

Audion Hub Manager - install notes

Main build paths

Recommended

Run:

```
builder_main.cmd
```

or directly:

```
install\Build_Portable_Env_Build.cmd
```

This is the main CMD build script.

Optional PowerShell route

Run:

```
install\Build_Portable_Env.cmd
```

This is a thin wrapper for the same-name `Build_Portable_Env.ps1`.

The wrapper looks for PowerShell in:

1. `system_core\powershell\pwsh.exe`
2. `pwsh.exe` in `PATH`
3. `powershell.exe` in `PATH`

Optional portable PowerShell installer

Portable PowerShell is not required by the Python project core. It is useful for fully portable GUI picker dialogs and build helper scripts.

From `builder_main.cmd`, choose the fixed numeric entry:

```
[04] POWERSHELL
```

or run directly:

```
install\Install-Portable-PowerShell.cmd
```

The installer downloads the latest Windows x64 PowerShell ZIP from GitHub and cleanly replaces:

```
system_core\powershell\
```

GUI picker dialogs prefer this portable `pwsh.exe` when it exists, then fall back to `pwsh.exe` from `PATH`, then to built-in `powershell.exe`.

The installer uses system `pwsh.exe` / `powershell.exe` as the updater when possible, so it can safely replace the target portable PowerShell tree. FZF is refreshed the same way: the latest GitHub release replaces only `system_core\fzf.exe`.

Install/update CMD scripts intentionally finish with `pause`. This keeps the resolved package version, download URL, success message, or error visible after a manual run.

BLAKE3 install / verify

BLAKE3 is listed in `install\requirements_full.in`, so the normal Python environment build installs it together with the rest of the runtime packages.

For repair or explicit refresh, use:

```
[05] BLAKE3 INSTALL
```

This runs `install\Install-BLAKE3.cmd`. It installs or updates `blake3` into the portable `runtime\` from local `wheelhouse\` when a BLAKE3 wheel is present; if no local wheel exists, it falls back to online `pip install --upgrade blake3`.

Builder step `[05]` is install/maintenance only. BLAKE3 diagnostics belong to the normal verification route:

```
[71] VERIFY / DOCTOR
```

Portable flow

1. Create folders
2. Resolve and download latest Python Embedded `3.12.x` ZIP
3. Extract to `runtime\`
4. Enable `import site` in `python3<minor>._pth`
5. Download `get-pip.py`
6. Install build bootstrap (`setuptools`, `wheel`, `packaging`)
7. Rebuild local `wheelhouse\` as installable wheels (`.gitkeep` is preserved, stale wheels/source archives are removed first)
8. Install packages into portable runtime from local `wheelhouse\`
9. Verify with `system_core\doctor.py` and `system_core\ui_nicegui\app.py --smoke`
10. Optionally create a release ZIP in `release\`

Offline flow

If `runtime\` and `wheelhouse\` are already populated, run:

```
install\install_portable_offline.cmd
```

Then verify with:

```
install\verify_portable_env.cmd
```

Release licensing

Third-party notices and license files are generated from the finalized staged release contents during `make_release_archive.cmd`. They are no longer generated during routine environment build/install steps.

Current Builder Order And Dependency Hygiene

`builder_main.cmd` uses fixed numeric entries. Keep the bootstrap order stable: `[01] PYTHON ENV CMD`, `[02] PYTHON ENV PS`, `[03] FZF`, `[04] POWERSHELL`, then project-specific payload

installers and one-time maintenance/diagnostic actions below.

Current builder install/maintenance map:

```
[01] PYTHON ENV CMD
[02] PYTHON ENV PS
[03] FZF
[04] POWERSHELL
[05] BLAKE3 INSTALL
[09] PORTABLE OFFLINE
[70] CLEAN INSTALL CACHE
[71] VERIFY / DOCTOR
[72] CMD ENCODING CHECK
[74] COLLECT RELEASE LICENSES
[75] PRUNE STALE LICENSE FOLDERS
[76] DEDUPLICATE LICENSE FILES
[77] MAKE RELEASE ARCHIVE
[90] PROJECT LAUNCHER
[95] OPEN install
[96] OPEN runtime
[97] OPEN wheelhouse
[98] OPEN licenses
[99] OPEN release
[00] EXIT
```

Project-specific payload entries before diagnostics:

No project-specific external payload installer before diagnostics.

Dependency hygiene rules:

- Python Embedded tracks the latest `3.12.x`; do not pin a concrete patch version in docs or scripts.
- Use the active embedded Python `_pth` file for path edits; do not hard-code a concrete filename.
- Bootstrap installs must include `setuptools`, `wheel`, and `packaging` before building or installing project wheels.
- `runtime\`, `wheelhouse\`, `system_core\powershell\`, `system_core\fzf.exe`, browser payloads, and external tool folders are reproducible payloads. Install/update scripts may cleanly replace only their owned targets.
- GPL or unknown-license external tools are explicit install/update payloads. Prefer GUI install buttons where the project exposes them, or fixed builder entries otherwise; do not silently bundle them as default source contents.

- BLAKE3 is a Python runtime dependency, not a separate external binary. Keep it in `requirements_full.in`; use builder step `[05] BLAKE3 INSTALL` only for explicit repair/update. Use `[71] VERIFY / DOCTOR` for diagnostics.
- `install\Clean-Install-Cache.cmd` / `.ps1` is the general install-cache cleanup. It removes transient `install\download\` artifacts (preserving `.gitkeep`, `get-pip.py`, and `7z*-extra.7z`), exact installer staging dirs `system_core\_pwsh_tmp` / `system_core\_fzf_tmp`, and Python bytecode caches outside runtime, wheelhouse, and user-data zones.
- `cleanup_project.cmd` is a separate source/release cleanup tool. It can remove runtime payloads and user-output zones after explicit confirmation; do not describe it as the general install-cache cleaner and do not wire it into install flow.